

Week 5 Structured Notes: 3D and 4D Arrays

Topic Overview

Week 5 explains how arrays with more than two dimensions are stored and calculated in memory.

In Week 4, you learned that a 2D array looks like a table, but memory stores it as one straight line. Week 5 continues that same idea.

A 3D or 4D array may look like blocks, layers, or cubes to humans, but inside memory, it is still stored as a single linear sequence.

This week mainly covers:

1. Review of 1D and 2D memory formulas
2. Meaning of 3D arrays
3. 3D row-major formula
4. 3D column-major formula
5. How to build 3D formulas logically
6. Meaning of 4D arrays
7. 4D row-major formula
8. 4D column-major formula
9. General n-dimensional array logic
10. Horner's Rule
11. Common mistakes
12. Practice examples
13. Checkpoint questions

1. What You Should Know Before Week 5

Before studying Week 5, you should already understand these ideas.

From Week 1

You should know that:

- A variable stores one value.
- An array stores many values under one name.
- Array indexing in C and C++ starts from 0.
- A loop is used to visit array elements.
- Array elements are stored side by side in memory.
- Accessing one element is fast.

Example:

```
int A[5] = {2, 4, 6, 8, 10};
```

Logical view:

Index:	0	1	2	3	4
Value:	2	4	6	8	10

If one integer takes 4 bytes and the base address is 200:

```
A[0] -> 200  
A[1] -> 204  
A[2] -> 208  
A[3] -> 212  
A[4] -> 216
```

From Week 2

You should know that:

- Stack arrays are fixed-size arrays.
- Heap arrays are created dynamically.
- A pointer stores a memory address.
- `malloc` creates heap memory in C.
- `free` releases heap memory in C.
- `new[]` creates a heap array in C++.
- `delete[]` releases a heap array in C++.

From Week 3

You should know the 1D array address formula:

```
Address of A[i] = L0 + (i * w)
```

Where:

```
L0 = base address  
i  = index  
w  = word size
```

Meaning:

```
Start at the base address.  
Skip i elements.  
Multiply by word size.
```

From Week 4

You should know the 2D row-major formula:

```
Address of A[i][j] = L0 + ((i * N) + j) * w
```

Where:

```
N = total number of columns
```

You should also know the 2D column-major formula:

```
Address of A[i][j] = L0 + ((j * M) + i) * w
```

Where:

```
M = total number of rows
```

Most important Week 4 idea:

```
A 2D array looks like a table, but memory stores it as one straight line.
```

2. Main Idea of Week 5

Week 5 is about extending 2D memory mapping into 3D, 4D, and general n-dimensional arrays.

The main question is always:

How many elements should we skip before reaching the target element?

After finding the number of skipped elements, the address is always calculated like this:

Address = Base address + skipped elements * word size

So the main formula idea is:

Address = L0 + offset * w

Where:

L0 = base address
offset = number of elements skipped
w = word size

Important:

3D and 4D arrays are not stored as real cubes in memory.
They are still stored as one straight line.

3. What Is a 3D Array?

A 3D array is an array with three indexes.

Example:

```
int A[2][3][4];
```

This means:

2 blocks
Each block has 3 rows
Each row has 4 columns

So:

```
A[2][3][4]
```

means:

```
First dimension size = 2  
Second dimension size = 3  
Third dimension size = 4
```

An element inside this array is written as:

```
A[I][J][K]
```

Where:

```
I = first index  
J = second index  
K = third index
```

4. Why We Use L, M, N and I, J, K

For 3D formulas, it is safer to use general names instead of names like layer, row, and column.

Use this notation:

```
Array dimensions: A[L][M][N]  
Target element:  A[I][J][K]
```

Meaning:

```
L, M, N = dimension sizes  
I, J, K = target indexes
```

Important difference:

L, M, N tell the size of the array.
I, J, K tell the position of the element.

Example:

```
int A[2][3][4];
```

Here:

```
L = 2  
M = 3  
N = 4
```

If the target is:

```
A[1][2][3]
```

Then:

```
I = 1  
J = 2  
K = 3
```

5. Row-Major Order in 3D

Row-major order means memory is filled from left to right.

For:

```
A[L][M][N]
```

Target:

```
A[I][J][K]
```

The 3D row-major formula is:

$$\text{Address} = L0 + ((I * M * N) + (J * N) + K) * W$$

Where:

L0 = base address
I = first index
J = second index
K = third index
M = second dimension size
N = third dimension size
W = word size

5.1 Understanding the 3D Row-Major Formula

Formula:

$$\text{Address} = L0 + ((I * M * N) + (J * N) + K) * W$$

Breakdown:

$$I * M * N$$

This skips full blocks before the target block.

$$J * N$$

This skips full rows inside the current block.

$$K$$

This moves inside the current row.

So:

$$\text{Skipped elements} = I * M * N + J * N + K$$

Then:

$$\text{Address} = L0 + \text{skipped elements} * W$$

6. Example: 3D Row-Major Address Calculation

Given:

Array: A[2][3][4]
Target: A[1][2][3]
Base address L0 = 1000
Word size W = 4

Identify dimensions:

L = 2
M = 3
N = 4

Identify indexes:

I = 1
J = 2
K = 3

Formula:

$$\text{Address} = L0 + ((I * M * N) + (J * N) + K) * W$$

Substitute values:

$$\text{Address} = 1000 + ((1 * 3 * 4) + (2 * 4) + 3) * 4$$

Calculate:

$$\begin{aligned}\text{Address} &= 1000 + (12 + 8 + 3) * 4 \\ \text{Address} &= 1000 + 23 * 4\end{aligned}$$


```
Address = 1000 + 92  
Address = 1092
```

Answer:

```
Address of A[1][2][3] = 1092
```

7. Column-Major Order in 3D

Column-major order means memory is filled from right to left.

For:

```
A[L][M][N]
```

Target:

```
A[I][J][K]
```

The 3D column-major formula is:

```
Address = L0 + ((K * L * M) + (J * L) + I) * W
```

Where:

```
L0 = base address  
I  = first index  
J  = second index  
K  = third index  
L  = first dimension size  
M  = second dimension size  
W  = word size
```

7.1 Understanding the 3D Column-Major Formula

Formula:

$$\text{Address} = L0 + ((K * L * M) + (J * L) + I) * W$$

Breakdown:

$$K * L * M$$

This skips earlier third-dimension groups.

$$J * L$$

This skips earlier second-dimension groups.

$$I$$

This moves inside the current group.

So:

$$\text{Skipped elements} = K * L * M + J * L + I$$

Then:

$$\text{Address} = L0 + \text{skipped elements} * W$$

8. Example: 3D Column-Major Address Calculation

Given:

Array: A[2][3][4]
Target: A[1][2][3]
Base address L0 = 1000
Word size W = 4

Identify dimensions:

L = 2
M = 3
N = 4

Identify indexes:

I = 1
J = 2
K = 3

Formula:

$$\text{Address} = L0 + ((K * L * M) + (J * L) + I) * W$$

Substitute values:

$$\text{Address} = 1000 + ((3 * 2 * 3) + (2 * 2) + 1) * 4$$

Calculate:

$$\begin{aligned}\text{Address} &= 1000 + (18 + 4 + 1) * 4 \\ \text{Address} &= 1000 + 23 * 4 \\ \text{Address} &= 1092\end{aligned}$$

Answer:

Address of A[1][2][3] = 1092

Important note:

In this example, row-major and column-major gave the same answer by coincidence. They will not always be the same.

9. Row-Major vs Column-Major in 3D

Feature	Row-Major	Column-Major
Direction	Left to right	Right to left
Formula direction	Start with I	Start with K
3D formula	$IMN + J*N + K$	$KLM + J*L + I$
Common in C/C++ built-in arrays	Yes	No
Main idea	Later dimensions are multiplied	Earlier dimensions are multiplied

Simple memory aid:

Row-major: move left to right.
Column-major: move right to left.

10. Easy Way to Build 3D Formulas

10.1 3D Row-Major Rule

For:

$A[L][M][N]$
 $A[I][J][K]$

Move left to right.

Write:

I gets $M * N$
J gets N
K gets nothing

So the skipped elements are:

$$I * M * N + J * N + K$$

Full formula:

$$\text{Address} = L0 + (I * M * N + J * N + K) * W$$

10.2 3D Column-Major Rule

For:

$$\begin{array}{l} A[L][M][N] \\ A[I][J][K] \end{array}$$

Move right to left.

Write:

K gets $L * M$
J gets L
I gets nothing

So the skipped elements are:

$$K * L * M + J * L + I$$

Full formula:

$$\text{Address} = L0 + (K * L * M + J * L + I) * W$$

11. What Is a 4D Array?

A 4D array has four indexes.

Example:

```
int A[d1][d2][d3][d4];
```

Target element:

```
A[i1][i2][i3][i4]
```

Meaning:

```
d1, d2, d3, d4 = dimension sizes  
i1, i2, i3, i4 = target indexes
```

A 4D array may look difficult, but it follows the same pattern as 3D arrays.

The main idea is still:

```
Address = L0 + skipped elements * word size
```

12. 4D Row-Major Formula

For:

```
A[d1][d2][d3][d4]
```

Target:

```
A[i1][i2][i3][i4]
```

Row-major means moving left to right.

Formula:

```
Address = L0 + (  
    i1 * d2 * d3 * d4 +  
    i2 * d3 * d4 +  
    i3 * d4 +
```

```
i4  
) * w
```

12.1 Understanding the 4D Row-Major Formula

Formula:

```
i1*d2*d3*d4 + i2*d3*d4 + i3*d4 + i4
```

Breakdown:

```
i1 * d2 * d3 * d4
```

This skips full first-dimension groups.

```
i2 * d3 * d4
```

This skips full second-dimension groups.

```
i3 * d4
```

This skips full third-dimension groups.

```
i4
```

This moves inside the last dimension.

So:

```
Skipped elements = i1*d2*d3*d4 + i2*d3*d4 + i3*d4 + i4
```

Then:

```
Address = L0 + skipped elements * w
```

13. Example: 4D Row-Major Address Calculation

Given:

```
Array: A[2][3][4][5]
Target: A[1][2][3][4]
Base address L0 = 1000
Word size w = 4
```

Identify dimensions:

```
d1 = 2
d2 = 3
d3 = 4
d4 = 5
```

Identify indexes:

```
i1 = 1
i2 = 2
i3 = 3
i4 = 4
```

Formula:

```
Address = L0 + (
    i1*d2*d3*d4 +
    i2*d3*d4 +
    i3*d4 +
    i4
) * w
```

Substitute values:

```
Address = 1000 + (
    1*3*4*5 +
    2*4*5 +
    3*5 +
    4
) * 4
```


Calculate:

```
Address = 1000 + (60 + 40 + 15 + 4) * 4
Address = 1000 + 119 * 4
Address = 1000 + 476
Address = 1476
```

Answer:

```
Address of A[1][2][3][4] = 1476
```

14. 4D Column-Major Formula

For:

```
A[d1][d2][d3][d4]
```

Target:

```
A[i1][i2][i3][i4]
```

Column-major means moving right to left.

Formula:

```
Address = L0 + (
    i4 * d1 * d2 * d3 +
    i3 * d1 * d2 +
    i2 * d1 +
    i1
) * w
```

14.1 Understanding the 4D Column-Major Formula

Formula:

$$i4*d1*d2*d3 + i3*d1*d2 + i2*d1 + i1$$

Breakdown:

$$i4 * d1 * d2 * d3$$

This skips earlier fourth-dimension groups.

$$i3 * d1 * d2$$

This skips earlier third-dimension groups.

$$i2 * d1$$

This skips earlier second-dimension groups.

$$i1$$

This moves inside the first dimension.

So:

$$\text{Skipped elements} = i4*d1*d2*d3 + i3*d1*d2 + i2*d1 + i1$$

Then:

$$\text{Address} = L0 + \text{skipped elements} * w$$

15. Example: 4D Column-Major Address Calculation

Given:

Array: A[2][3][4][5]
Target: A[1][2][3][4]

Base address L0 = 1000
Word size w = 4

Identify dimensions:

d1 = 2
d2 = 3
d3 = 4
d4 = 5

Identify indexes:

i1 = 1
i2 = 2
i3 = 3
i4 = 4

Formula:

```
Address = L0 + (
    i4*d1*d2*d3 +
    i3*d1*d2 +
    i2*d1 +
    i1
) * w
```

Substitute values:

```
Address = 1000 + (
    4*2*3*4 +
    3*2*3 +
    2*2 +
    1
) * 4
```

Calculate:

```
Address = 1000 + (96 + 18 + 4 + 1) * 4
Address = 1000 + 119 * 4
Address = 1476
```

Answer:

Address of `A[1][2][3][4]` = 1476

Important note:

This example gives the same answer as row-major by coincidence.
Different target indexes can give different answers.

16. General n-Dimensional Logic

The same idea works for arrays with any number of dimensions.

16.1 Row-Major General Rule

Row-major order moves from left to right.

Rule:

Each index is multiplied by all dimension sizes after it.
The last index stands alone.

For:

`A[d1][d2][d3][d4]`
`A[i1][i2][i3][i4]`

Row-major skipped elements:

$i1*d2*d3*d4 + i2*d3*d4 + i3*d4 + i4$

General memory aid:

Row-major = multiply by dimensions on the right.

16.2 Column-Major General Rule

Column-major order moves from right to left.

Rule:

Each index is multiplied by all dimension sizes before it.
The first index stands alone.

For:

$A[d1][d2][d3][d4]$
 $A[i1][i2][i3][i4]$

Column-major skipped elements:

$i4*d1*d2*d3 + i3*d1*d2 + i2*d1 + i1$

General memory aid:

Column-major = multiply by dimensions on the left.

17. Horner's Rule

Horner's Rule is an optimization method.

It rewrites a formula so the computer does fewer repeated multiplications.

The normal 4D row-major expression is:

$i1*d2*d3*d4 + i2*d3*d4 + i3*d4 + i4$

This works, but it repeats many multiplications.

Horner-style form:

```
Result = i4 + d4 * (i3 + d3 * (i2 + i1 * d2))
```

17.1 Beginner Explanation of Horner's Rule

Normal expression:

```
i1*d2*d3*d4  
i2*d3*d4  
i3*d4  
i4
```

The computer has to calculate several multiplication groups.

Horner-style expression:

```
Result = i4 + d4 * (i3 + d3 * (i2 + i1 * d2))
```

This builds the result step by step.

Simple meaning:

```
Instead of calculating many big products separately,  
Horner's Rule reuses smaller calculations.
```

Why it matters:

```
Compilers try to reduce repeated work.  
Horner's Rule helps optimize address calculation.
```

18. Formula Quick Reference

1D Formula

```
Address of A[i] = L0 + (i * w)
```

2D Row-Major Formula

Address of $A[i][j] = L_0 + ((i * N) + j) * w$

Where:

N = total columns

2D Column-Major Formula

Address of $A[i][j] = L_0 + ((j * M) + i) * w$

Where:

M = total rows

3D Row-Major Formula

Address = $L_0 + ((I * M * N) + (J * N) + K) * W$

3D Column-Major Formula

Address = $L_0 + ((K * L * M) + (J * L) + I) * W$

4D Row-Major Formula

Address = $L_0 + ($
 $i_1 * d_2 * d_3 * d_4 +$
 $i_2 * d_3 * d_4 +$
 $i_3 * d_4 +$

```
    i4  
  ) * w
```

4D Column-Major Formula

```
Address = L0 + (  
    i4*d1*d2*d3 +  
    i3*d1*d2 +  
    i2*d1 +  
    i1  
  ) * w
```

19. Memory Aid: How the Formulas Grow

1D

```
A[i]  
Skip i elements.
```

Formula:

```
i
```

2D Row-Major

```
A[i][j]  
Skip i full rows, then j columns.
```

Formula:

```
i * columns + j
```


3D Row-Major

`A[I][J][K]`

Skip I full blocks, then J full rows, then K columns.

Formula:

`I*M*N + J*N + K`

4D Row-Major

`A[i1][i2][i3][i4]`

Move left to right.

Formula:

`i1*d2*d3*d4 + i2*d3*d4 + i3*d4 + i4`

Column-Major Shortcut

For column-major, reverse the direction.

Move right to left.

20. Common Week 5 Mistakes

Mistake 1: Memorizing Without Understanding

Bad approach:

Trying to memorize every formula separately.

Better approach:

Build the formula from the dimension order.

Mistake 2: Mixing Up Dimensions and Indexes

For:

```
A[L][M][N]  
A[I][J][K]
```

These are not the same:

```
L, M, N = sizes  
I, J, K = positions
```

Mistake 3: Forgetting Word Size

This gives only skipped elements:

```
I*M*N + J*N + K
```

The final address still needs:

```
L0 + skipped_elements * W
```

Mistake 4: Confusing Row-Major and Column-Major

Row-major:

```
Move left to right.
```

Column-major:

Move right to left.

Mistake 5: Thinking 3D or 4D Arrays Are Stored as Physical Cubes

Wrong idea:

A 3D array is stored as a real cube in memory.

Correct idea:

A 3D or 4D array may look like a cube or block logically, but physical memory is still one straight line.

Mistake 6: Using the Wrong Dimension in the Formula

For 3D row-major:

Correct: $I * M * N + J * N + K$

Wrong examples:

$I * L * M + J * M + K$
 $I * N + J * M + K$

Why wrong?

Because in row-major order, each index is multiplied by the dimensions after it.

21. Practice Example 1: 3D Row-Major

Given:

$A[3][4][5]$
Target = $A[2][1][3]$

$$\begin{aligned}L_0 &= 500 \\ W &= 2\end{aligned}$$

Identify dimensions:

$$\begin{aligned}L &= 3 \\ M &= 4 \\ N &= 5\end{aligned}$$

Identify indexes:

$$\begin{aligned}I &= 2 \\ J &= 1 \\ K &= 3\end{aligned}$$

Formula:

$$\text{Address} = L_0 + ((I * M * N) + (J * N) + K) * W$$

Substitute:

$$\text{Address} = 500 + ((2 * 4 * 5) + (1 * 5) + 3) * 2$$

Calculate:

$$\begin{aligned}\text{Address} &= 500 + (40 + 5 + 3) * 2 \\ \text{Address} &= 500 + 48 * 2 \\ \text{Address} &= 596\end{aligned}$$

Answer:

$$596$$

22. Practice Example 2: 3D Column-Major

Given:

```
A[3][4][5]
Target = A[2][1][3]
L0 = 500
W = 2
```

Identify dimensions:

```
L = 3
M = 4
N = 5
```

Identify indexes:

```
I = 2
J = 1
K = 3
```

Formula:

```
Address = L0 + ((K*L*M) + (J*L) + I) * W
```

Substitute:

```
Address = 500 + ((3*3*4) + (1*3) + 2) * 2
```

Calculate:

```
Address = 500 + (36 + 3 + 2) * 2
Address = 500 + 41 * 2
Address = 582
```

Answer:

```
582
```

Comparison:

```
Row-major address    = 596
Column-major address = 582
```

This proves that the same logical element can have different addresses depending on the storage order.

23. Practice Example 3: 4D Row-Major

Given:

```
A[2][3][4][5]
Target = A[1][1][2][3]
L0 = 1000
w = 4
```

Identify dimensions:

```
d1 = 2
d2 = 3
d3 = 4
d4 = 5
```

Identify indexes:

```
i1 = 1
i2 = 1
i3 = 2
i4 = 3
```

Formula:

```
Address = L0 + (
    i1*d2*d3*d4 +
    i2*d3*d4 +
    i3*d4 +
    i4
) * w
```

Substitute:

```
Address = 1000 + (  
    1*3*4*5 +  
    1*4*5 +  
    2*5 +  
    3  
) * 4
```

Calculate:

```
Address = 1000 + (60 + 20 + 10 + 3) * 4  
Address = 1000 + 93 * 4  
Address = 1372
```

Answer:

1372

24. Week 5 Key Terms

Term	Simple Meaning
3D array	An array with three indexes
4D array	An array with four indexes
Dimension	Size of one direction in an array
Index	Position of the target element
Base address	Starting address of the array
Word size	Number of bytes used by one element
Offset	Number of elements skipped from the beginning
Row-major order	Memory order that moves left to right
Column-major order	Memory order that moves right to left
Linear memory	Memory arranged as one straight sequence
Horner's Rule	Optimization method that reduces repeated multiplication
n-dimensional array	An array with any number of dimensions

25. Week 5 Summary

By the end of Week 5, you should understand that:

- 3D arrays have three indexes.
- 4D arrays have four indexes.
- Multidimensional arrays may look like blocks, but memory stores them linearly.
- Address calculation always starts with the base address.
- The formula first finds how many elements to skip.
- The number of skipped elements is multiplied by word size.
- Row-major order moves from left to right.
- Column-major order moves from right to left.
- In 3D row-major order, the formula is:

$$\text{Address} = L0 + ((I * M * N) + (J * N) + K) * W$$

- In 3D column-major order, the formula is:

$$\text{Address} = L0 + ((K * L * M) + (J * L) + I) * W$$

- In 4D row-major order, the formula is:

$$\text{Address} = L0 + (i1 * d2 * d3 * d4 + i2 * d3 * d4 + i3 * d4 + i4) * w$$

- In 4D column-major order, the formula is:

$$\text{Address} = L0 + (i4 * d1 * d2 * d3 + i3 * d1 * d2 + i2 * d1 + i1) * w$$

- Horner's Rule reduces repeated multiplication.
- The best way to learn these formulas is to build them from dimension order.

26. Week 5 Checkpoint Questions

Try answering these without looking at the notes.

1. What is a 3D array?
2. What does `A[I][J][K]` mean?
3. In `A[L][M][N]`, what do `L`, `M`, and `N` mean?
4. In `A[I][J][K]`, what do `I`, `J`, and `K` mean?
5. What is the 3D row-major formula?
6. What is the 3D column-major formula?

7. In row-major order, do we move left to right or right to left?
 8. In column-major order, do we move left to right or right to left?
 9. What is a 4D array?
 10. What does `A[i1][i2][i3][i4]` mean?
 11. What is the 4D row-major formula?
 12. What is the 4D column-major formula?
 13. What is the difference between dimension size and index?
 14. Why do we multiply by word size?
 15. What is an offset?
 16. Why is memory still linear for 3D and 4D arrays?
 17. Why can row-major and column-major give different addresses?
 18. What does Horner's Rule improve?
 19. Why is memorizing formulas without understanding risky?
 20. What is the most important idea of Week 5?
-

27. Practice Tasks

Task 1: 3D Row-Major Calculation

Given:

```
A[2][3][4]
Target = A[1][1][2]
L0 = 200
W = 4
```

Find the address using row-major order.

Formula:

```
Address = L0 + ((I*M*N) + (J*N) + K) * W
```

Task 2: 3D Column-Major Calculation

Using the same data:

```
A[2][3][4]
Target = A[1][1][2]
```

```
L0 = 200
W = 4
```

Find the address using column-major order.

Formula:

```
Address = L0 + ((K*L*M) + (J*L) + I) * W
```

Task 3: Compare Row-Major and Column-Major

Given:

```
A[3][4][5]
Target = A[2][1][3]
L0 = 500
W = 2
```

Calculate:

1. Row-major address
2. Column-major address

Then explain why the answers are different.

Task 4: 4D Row-Major Calculation

Given:

```
A[2][3][4][5]
Target = A[1][1][2][3]
L0 = 1000
w = 4
```

Find the address using row-major order.

Task 5: Build the Formula Yourself

Write the row-major formula for:

```
A[a][b][c][d][e]  
Target = A[i1][i2][i3][i4][i5]
```

Hint:

```
Move left to right.  
Each index is multiplied by all dimensions after it.
```

28. Simple Final Memory Aid

```
Address = L0 + skipped_elements * word_size
```

For row-major:

```
Move left to right.  
Multiply each index by the dimensions after it.
```

For column-major:

```
Move right to left.  
Multiply each index by the dimensions before it.
```

3D row-major:

```
I*M*N + J*N + K
```

3D column-major:

```
K*L*M + J*L + I
```

4D row-major:

$$i_1*d_2*d_3*d_4 + i_2*d_3*d_4 + i_3*d_4 + i_4$$

4D column-major:

$$i_4*d_1*d_2*d_3 + i_3*d_1*d_2 + i_2*d_1 + i_1$$

Horner's Rule:

Rewrite formulas to reduce repeated multiplication.

Most important Week 5 idea:

Do not memorize blindly.
Build the formula from the dimension order.